Learning Body State from Pressure Sensor Data

Denoising → Representation Learning → Decision-Making on a Synthetic Pressure-Mattress Benchmark

---

What this is: an end-to-end project covering (1) a denoising autoencoder for raw pressure-sensor readings, (2) self-supervised representation learning evaluated under a few-shot labeling budget, with baselines and ablations, and (3) a contextual bandit for deciding which mattress zone to adjust under a hidden user comfort function.

Data: a programmatic synthetic pressure-map generator with a documented, explicit sensor noise model (code: `src/data.py`), used in place of the request-gated PMAP dataset.

Reproduce: `pip install -r requirements.txt && bash run_all.sh` — full pipeline runs in under a minute on CPU.

Motivation

A pressure-sensing mattress reports a 2D grid of raw taxel readings. Three properties make this a genuinely interesting ML problem rather than a plain classification task. First, the raw signal is noisy — resistive sensor arrays drop cells, spike under cross-talk, and quantize coarsely, so it is worth knowing how to clean the signal and how much that cleaning is actually worth. Second, labels are expensive — a real deployment might realistically get 5–10 labeled posture examples per class, which is exactly the regime where self-supervised pretraining is supposed to pay off. Third, sensing should drive action: the end goal of a smart mattress is a decision (which zone to relieve right now), made under a comfort function that is never directly observable — a contextual bandit problem. The three phases of this project map directly onto these three points.

Data

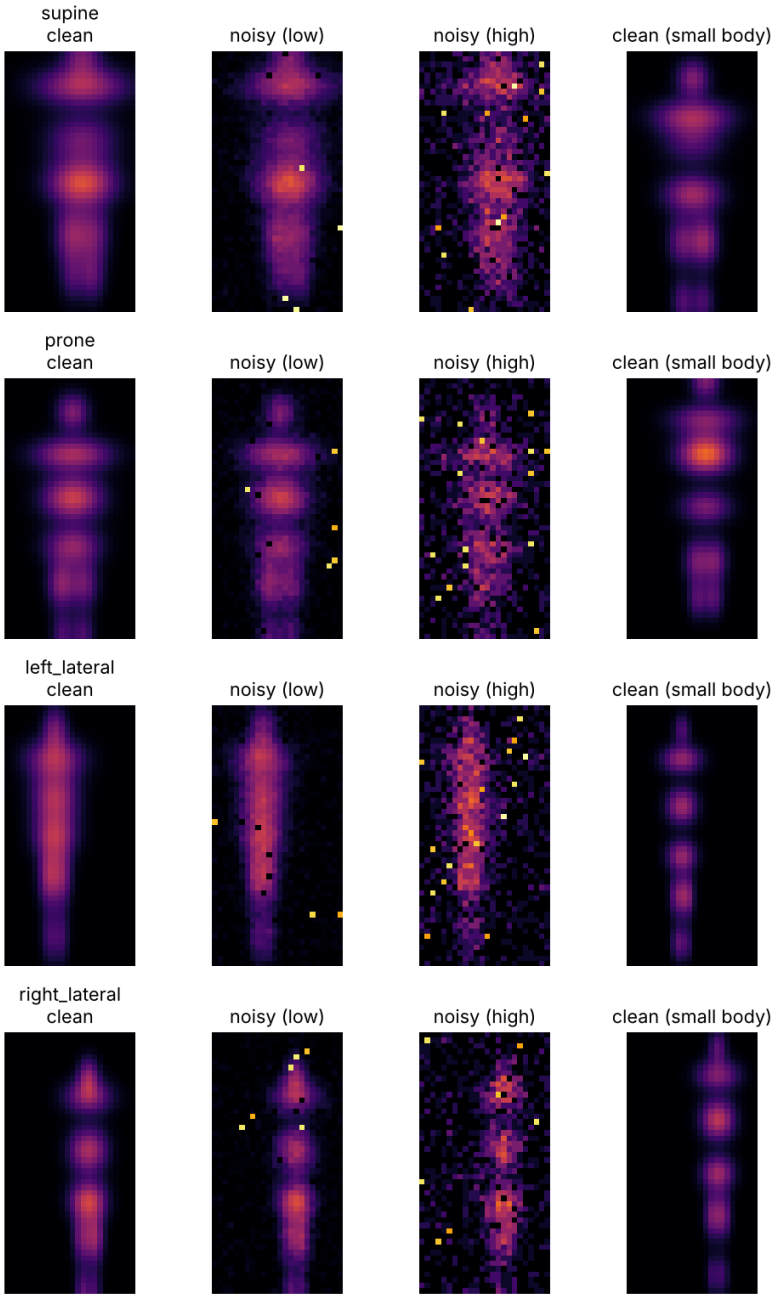
Choice: synthetic generator. The real PMAP dataset is human-subjects data and is request-gated; for fast iteration, a programmatic generator that preserves the structure of the real problem is more honest than stalling on data access, and it makes every noise assumption explicit and auditable.

The generator produces a 48×24 taxel grid. A body is a sum of anisotropic Gaussian ‘blobs’ (head, shoulders, chest, pelvis, thighs, calves). Posture is the label (`supine, prone, left_lateral, right_lateral`): lateral postures are shifted and narrower (side-lying footprint), while supine vs. prone — the harder pair — differ only in the relative intensity of the chest vs. pelvis blob. Body size (small/medium/large, plus continuous jitter) and a global random placement offset are both generated but withheld from the model as nuisance variables, used later for failure analysis.

Documented sensor noise model:

noisy = quantize(clip(clean + Gaussian(σ) + dead-cell dropout(p_{drop}) + salt spikes(p_{salt}), 0, 1), levels)

with σ, p_{drop}, p_{salt}, and quantization levels scaling together across three severities (low/medium/high, mixed uniformly over the dataset): σ 0.02→0.09, p_{drop} 1%→6%, p_{salt} 0.2%→1.2%, levels 64→16.



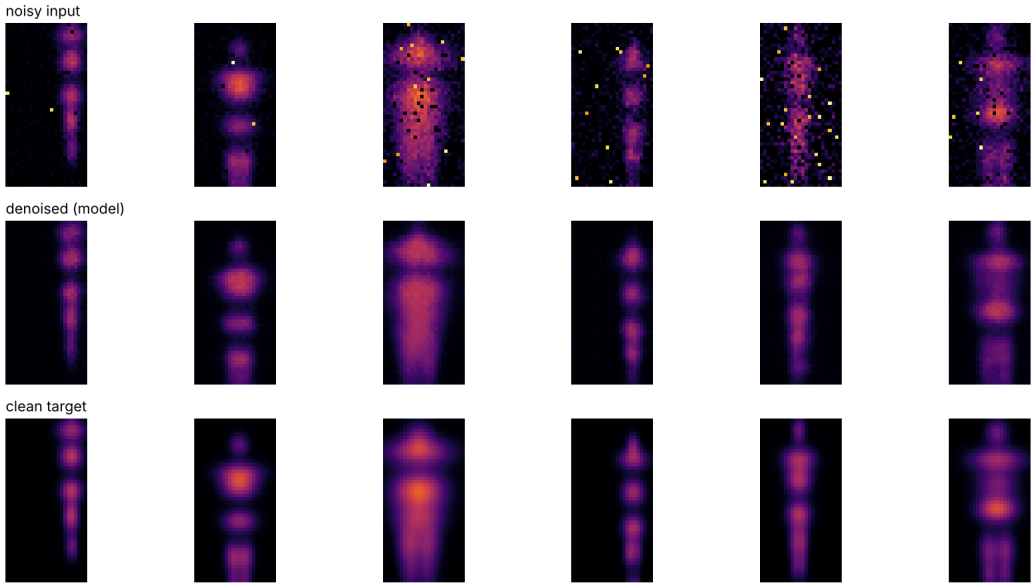
Clean / noisy (low) / noisy (high) / clean-small-body renders for each posture class.

Phase 1 — Denoising

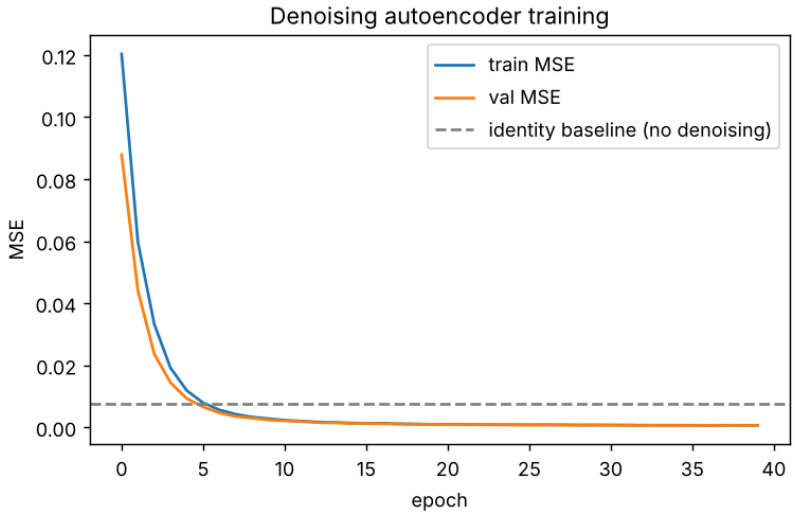
A small conv encoder-decoder (3 stride-2 conv blocks each way, 64-channel bottleneck) trained with MSE to map noisy → clean. 40 epochs, 2000 train / 500 validation pairs, ~14s on CPU.

	MSE	PSNR
Identity baseline (do nothing)	0.00769	21.1 dB
Denoising autoencoder	0.00063	32.0 dB

91.8% reduction in reconstruction MSE vs. doing nothing.



Noisy input / model denoised output / clean target, six held-out validation samples.



Training curve against the identity-baseline MSE floor.

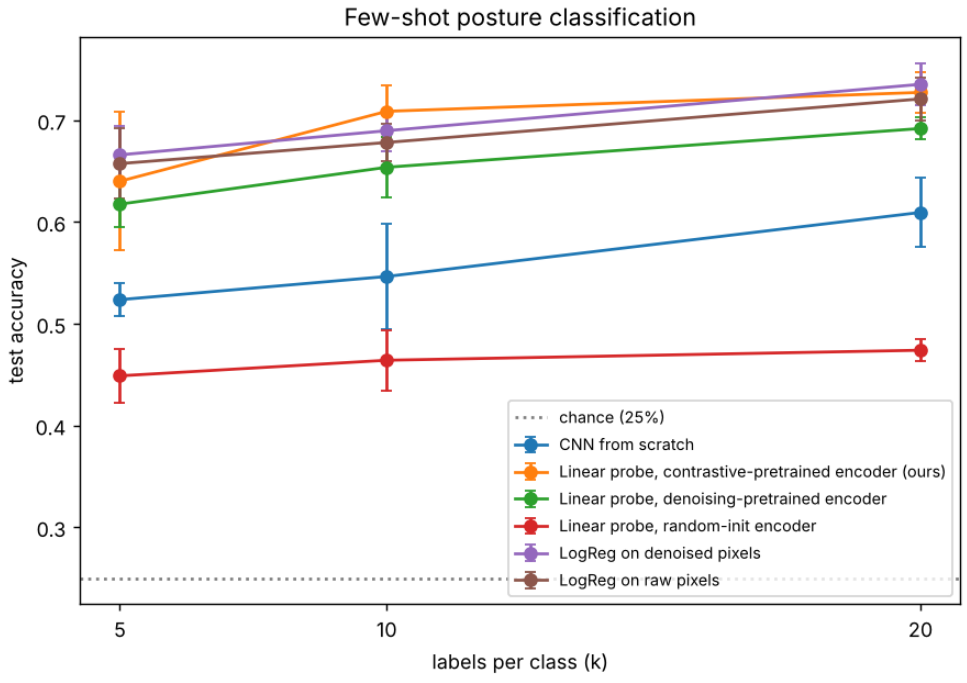
Phase 2 — Representation Learning + Few-Shot Classification

The same backbone architecture is pretrained two ways on *unlabeled* data, so only the pretraining signal differs downstream, not the architecture: **contrastive** (SimCLR-style NT-Xent, temperature 0.3, augmentations = noise/translation/dropout/intensity-scale — deliberately no horizontal flip, since flipping would swap left/right lateral labels), and **denoising** (the Phase 1 autoencoder's backbone, reused directly).

Evaluated for k ∈ {5, 10, 20} labels/class (4 classes), averaged over 8 random label draws, against a held-out test set, six methods:

method	k=5	k=10	k=20
Linear probe, random-init encoder (floor)	44.9%	46.4%	47.4%
CNN trained from scratch (k-shot only)	52.4%	54.6%	60.9%
Linear probe, denoising-pretrained encoder	61.7%	65.4%	69.2%
LogReg on raw noisy pixels	65.7%	67.8%	72.1%
LogReg on denoised pixels	66.6%	69.0%	73.5%
Linear probe, contrastive-pretrained encoder (ours)	64.0%	70.9%	72.7%

(chance = 25%; full numbers with std in reports/results/fewshot_summary.json)



Few-shot test accuracy vs. labels per class, mean ± std over 8 draws.

Step 3 — Evaluating Like a Researcher

Ablations

- **Does pretraining help, and by how much?** Yes, decisively, against the only fair control (same architecture, random weights): the random-init linear probe sits at 45–47% regardless of k, while either pretraining method lifts it to 62–73% — a +17 to +26 point jump.
- **Does denoising help downstream accuracy?** Yes, modestly and consistently: LogReg on denoised pixels beats LogReg on raw noisy pixels at every k (+0.9, +1.2, +1.4 points). A real but small effect.
- **Does the CNN-from-scratch baseline justify its cost?** No — it is the worst non-floor method at every k, confirming that training a full conv net end-to-end on 20–80 total labels overfits badly compared to fitting a linear head on good frozen features.

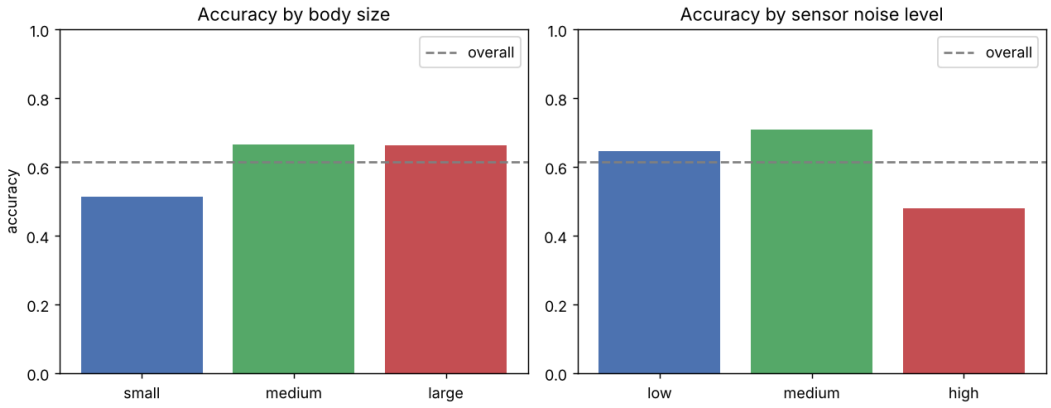
Honest surprise: raw pixels are a strong baseline

The first version of this project used perfectly-centered bodies, and raw-pixel logistic regression hit **99%+ accuracy with only 10 labels/class** — beating every pretrained representation by 20+ points. That is not a success story, it is a sign the synthetic task was too easy: with no nuisance variation, posture differences are a handful of cleanly linearly-separable pixels, and 1152-dimensional logistic regression with 5–20 samples per class will happily overfit a perfect linear boundary in that regime (classic n■d). The fix, documented in `src/data.py`, was to add a **global random body placement offset per sample**, independent of posture — the subject is not perfectly centered on the mattress, which is true of real sensor data. That single change dropped raw-pixel LogReg from 99%→~70% and let the representation-learning story actually show up. **Lesson:** a synthetic benchmark that makes your proposed method look great by a huge margin is itself a result worth being suspicious of.

Even after the fix, raw-pixel LogReg stays *competitive* with the pretrained linear probes rather than clearly losing — on a 48×24, 4-class problem a 1152-dim linear boundary is a genuinely strong baseline. We report this rather than cherry-picking a setting that flatters the pretrained encoder.

Failure analysis

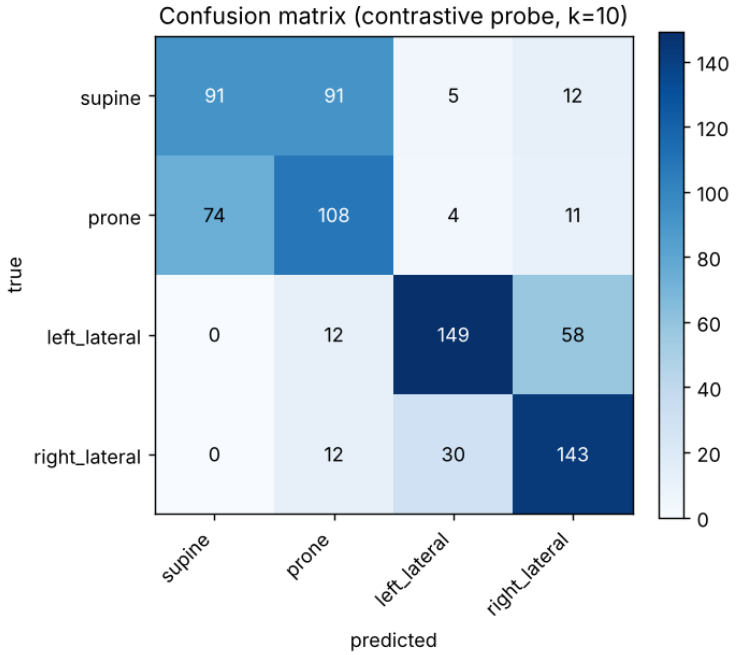
Contrastive probe, k=10, a single representative label draw, evaluated on the full 800-sample held-out test set (overall accuracy 61.4%):



Accuracy broken down by nuisance variables never given to the model: body size and sensor noise level.

• **Body size:** small bodies are the clear weak point (51.3% vs. 66.5% / 66.4% for medium/large) — smaller blobs produce weaker, more noise-fragile pressure signatures relative to a fixed sensor-noise magnitude.

• **Sensor noise level:** accuracy drops sharply at high noise (48.1%) vs. low/medium (64.5% / 70.9%) — a reminder that ‘works on clean sensors’ and ‘works in the field’ are different claims.

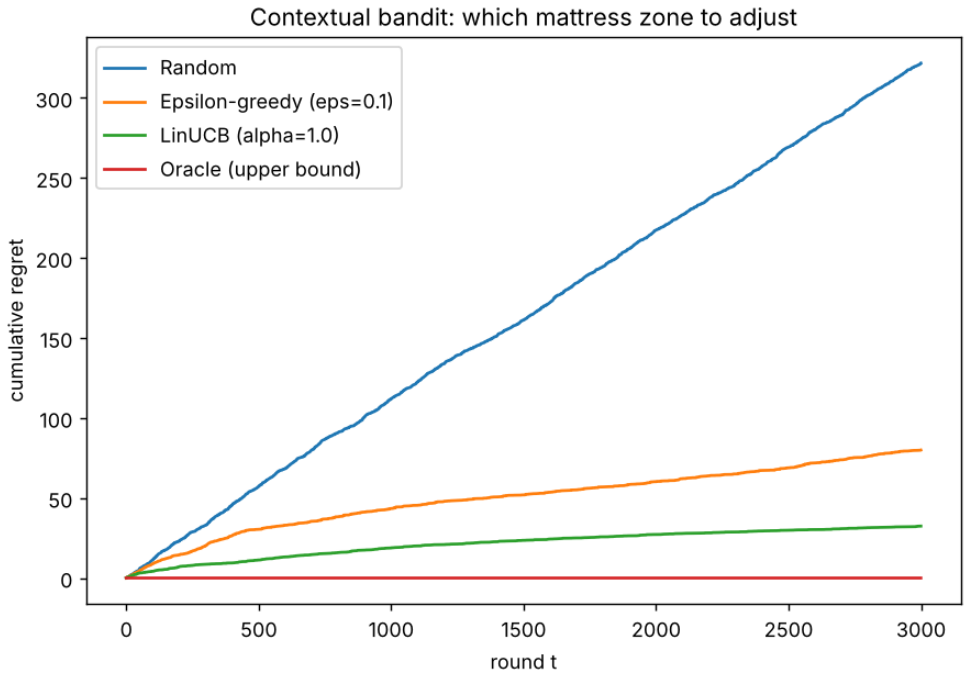


Supine and prone are confused with each other constantly, essentially never with either lateral posture; left/right lateral confuse each other far less. This matches the generative design exactly: lateral-vs-not is a gross, noise-robust spatial signal, while supine-vs-prone is a subtle chest/pelvis intensity ratio that noise and small body size both directly corrupt.

Phase 3 (Stretch) — Contextual Bandit for Zone Adjustment

The mattress is split into 5 zones (head, shoulders, lower-back/hips, thighs, calves). At each round the agent observes per-zone mean pressure as context and picks one zone to relieve. A simulated user has a hidden, fixed per-zone comfort-sensitivity vector θ (unknown to the agent): reward(zone z) = $\theta_z \cdot \text{pressure}_z$ + noise — a disjoint linear contextual bandit. Policies: random, epsilon-greedy (per-arm ridge regression, ε=0.1), LinUCB (disjoint, Li et al. 2010, α=1.0), and an oracle that knows θ exactly. T=3000 rounds.

policy	final cumulative regret	mean reward (last 500 rounds)
Random	321.8	0.139
Epsilon-greedy	80.0	0.222
LinUCB	32.5	0.239
Oracle (upper bound)	0.0	0.244



LinUCB's context-aware confidence bound closes the gap to oracle far faster than epsilon-greedy's undirected exploration. Regret is visibly sublinear (concave) for both learning policies and linear for random, exactly as theory predicts.

Limitations

• **Synthetic data, not real PMAP.** The generator is a reasonable stand-in for the spatial structure of the problem, but cannot capture real inter-subject anatomical variation, sensor nonlinearity/hysteresis, foam compression dynamics, or spatially-correlated noise.

• **Representation learning's advantage is real but modest, not dominant.** Plain logistic regression on (denoised) pixels is a strong baseline here; contrastive pretraining's clearest win is at k=10, not uniformly across all k. The honest claim is ‘reliably beats random-init features by a wide margin, competitive-to-best against simple baselines’, not ‘dominates’.

• **The bandit's comfort function is linear and stationary.** Real human comfort preferences likely drift over a night's sleep and are not perfectly linear in zone pressure; a real system would need non-stationary bandit methods.

• **Small grid, 4 posture classes,** chosen deliberately for fast CPU-only iteration; conclusions about model capacity should not be assumed to transfer to higher-resolution arrays or finer posture taxonomies without re-validation.

• **No cross-session sensor drift/calibration was modeled,** beyond the per-reading noise draw — a real deployment sees sensors age and drift over weeks, a different, slower-timescale problem than what's simulated here.

Future Work

• Validate against a real (even small, consented) pressure-mat dataset to check whether the synthetic-data conclusions actually transfer.

• Try a denoising + contrastive joint pretraining objective to see if the two signals are complementary rather than alternatives.

• Extend the bandit's reward model to be non-stationary and to richer action spaces (continuous adjustment amount, not just zone choice).

• Scale grid resolution and posture taxonomy once a real dataset is available, re-running the same baseline/ablation/failure-analysis protocol used here unchanged — it was built to be dataset-agnostic.